

Dokumentacja Techniczna Projektu

Program Do Wyznaczania Cyklu Hamiltona W Grafie

Julia Litawa

Maj 2026

Spis treści

1	Wstęp	2
2	Wymagania i Specyfikacja Projektu	2
2.1	Spełnione wymagania techniczne	2
3	Struktura Projektu	2
4	Reprezentacja Grafu	2
5	Znaczące Zmienne i Struktury Danych	3
6	Opis Funkcjonalności (API)	3
6.1	Obsługa programu – <code>main.cpp</code>	3
6.2	Obsługa grafu i cykli Hamiltona – <code>hamilton.cpp</code>	4
7	Instrukcja Obsługi i Kompilacja	4
7.1	Plik Makefile	4
7.2	Kompilacja i Uruchomienie	4

1 Wstęp

Celem projektu jest stworzenie aplikacji konsolowej w języku C++, umożliwiającej wykrywanie cyklu Hamiltona (czyli takiego, w którym każdy wierzchołek grafu odwiedzany jest tylko raz) w grafie. Aplikacja została zaprojektowana tak, aby umożliwić podawanie na wejście grafu, zarówno podając dane ręcznie za pomocą interfejsu konsolowego, jak i dane odczytane z pliku.

2 Wymagania i Specyfikacja Projektu

Projekt został zrealizowany zgodnie z narzuconą specyfikacją techniczną oraz wymaganiami funkcjonalnymi.

2.1 Spełnione wymagania techniczne

- **Język programowania:** C++.
- **Objętość kodu:** Projekt przekracza wymagane minimum 100 wierszy kodu. Plik `main.cpp` zawiera 39 wierszy kodu, `hamilton.cpp` 83 wiersze kodu, a `hamilton.h` 11 wierszy, co daje łącznie 133 wiersze kodu.
- **Styl kodu:** Zgodnie z wytycznymi, w projekcie nie użyto dyrektywy `using namespace std;`. Zamiast tego korzystano z jawnych nazw np. `std::cout`, co jasno wskazuje pochodzenie elementów i zapobiega kolizjom, zmniejszając ryzyko trudnych do wykrycia błędów.
- **Kompilacja:** Przygotowano plik `Makefile` umożliwiający automatyczną kompilację z wymaganymi flagami: `-Wall -std=c++11`.

3 Struktura Projektu

Kod źródłowy został podzielony na moduły, co ułatwia zarządzanie projektem i jego kompilację.

- `main.cpp` – Główna funkcja programu, obsługa wyboru sposobu wprowadzania pliku (1 - wczytaj ręcznie, 2 - wczytaj z pliku) i wypisywanie znalezionych cykli Hamiltona lub informacja o ich braku, program działa w pętli, z której wychodzi się, wybierając 3 na klawiaturze.
- `hamilton.h` – Plik nagłówkowy z deklaracjami funkcji.
- `hamilton.cpp` – Plik nagłówkowy z definicjami zadeklarowanych wcześniej funkcji.
- `Makefile` – Skrypt automatyzujący proces budowania aplikacji.

4 Reprezentacja Grafu

W programie graf reprezentowany jest za pomocą **macierzy sąsiedztwa**. Struktura grafu przechowywana jest w postaci dwuwymiarowego wektora typu `std::vector<std::vector<int>>`.

Element macierzy `graph[i][j]` określa istnienie połączenia pomiędzy wierzchołkami i oraz j . Wartość 1 oznacza istnienie krawędzi, natomiast wartość 0 wskazuje na jej brak.

Zastosowanie macierzy sąsiedztwa pozwala na szybkie sprawdzenie istnienia krawędzi pomiędzy dwoma wierzchołkami, co upraszcza implementację algorytmu wyszukiwania cyklu Hamiltona.

5 Znaczące Zmienne i Struktury Danych

W implementacji programu wykorzystano następujące istotne zmienne i struktury danych:

- **graph** – dwuwymiarowy wektor typu `std::vector<std::vector<int>>`, reprezentujący graf w postaci macierzy sąsiedztwa. Element `graph[i][j]` określa istnienie krawędzi pomiędzy wierzchołkami i i j .
- **path** – wektor typu `std::vector<int>` przechowujący aktualnie budowaną ścieżkę Hamiltona. Kolejne indeksy odpowiadają odwiedzonym wierzchołkom.
- **visited** – wektor logiczny typu `std::vector<bool>`, wykorzystywany do oznaczania odwiedzonych wierzchołków w trakcie działania algorytmu. Wartość `true` oznacza wierzchołek już odwiedzony.
- **cycles** – dwuwymiarowy wektor typu `std::vector<std::vector<int>>`, przechowujący wszystkie znalezione cykle Hamiltona.
- **n** – zmienna typu `int`, oznaczająca liczbę wierzchołków grafu.
- **pos** – zmienna typu `int`, określająca aktualną pozycję w ścieżce Hamiltona podczas działania funkcji rekurencyjnej.
- **v** – zmienna pomocnicza typu `int`, używana do iteracji po kolejnych wierzchołkach grafu.

6 Opis Funkcjonalności (API)

6.1 Obsługa programu – `main.cpp`

- **main**: Funkcja główna programu. Wyświetla menu wyboru sposobu wprowadzenia grafu. Użytkownik może wybrać ręczne podanie danych lub wczytanie grafu z pliku. Po poprawnym wczytaniu danych funkcja przekazuje graf do algorytmu wyszukiwania cykli Hamiltona.
- **wybor**: Zmienna sterująca działaniem programu. Na jej podstawie wykonywane jest odpowiednie wczytanie grafu: z klawiatury lub z pliku.
- **graf**: Główna struktura przechowująca graf w programie. Graf reprezentowany jest jako macierz sąsiedztwa typu `std::vector<std::vector<int>>`.

6.2 Obsługa grafu i cykli Hamiltona – `hamilton.cpp`

- **wczytajRecznie**: Umożliwia ręczne wprowadzenie grafu przez użytkownika. Najpierw pobierana jest liczba wierzchołków, a następnie macierz sąsiedztwa o wymiarach $n \times n$.
- **wczytajZPliku**: Odpowiada za wczytanie grafu z pliku tekstowego. Pierwsza liczba w pliku określa liczbę wierzchołków, a kolejne wartości tworzą macierz sąsiedztwa.
- **hamiltonCycle**: Funkcja inicjująca proces wyszukiwania cykli Hamiltona. Tworzy pomocnicze struktury danych: `path`, `visited` oraz `cycles`. Następnie uruchamia funkcję rekurencyjną odpowiedzialną za przeszukiwanie grafu.
- **hamiltonCycleUtil**: Funkcja rekurencyjna realizująca algorytm z nawrotami (*backtracking*). Sprawdza kolejne możliwe wierzchołki, buduje ścieżkę Hamiltona i cofa się, gdy dalsze przejście nie jest możliwe. Po odwiedzeniu wszystkich wierzchołków sprawdza, czy istnieje krawędź powrotna do wierzchołka początkowego.

7 Instrukcja Obsługi i Kompilacja

7.1 Plik Makefile

Zgodnie z wymaganiami projekt zawiera plik `Makefile`, który definiuje reguły kompilacji.

Listing 1: Zawartość pliku `Makefile`

```
CC = g++
CFLAGS = -Wall -std=c++11
OBJ = main.o hamilton.o
TARGET = cykl_hamiltona

all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJ)

main.o: main.cpp hamilton.h
    $(CC) $(CFLAGS) -c main.cpp

hamilton.o: hamilton.cpp hamilton.h
    $(CC) $(CFLAGS) -c hamilton.cpp

clean:
    rm -f *.o $(TARGET)
```

7.2 Kompilacja i Uruchomienie

Aby zbudować projekt przy użyciu narzędzia `make`, należy w terminalu (w katalogu projektu) wywołać polecenie:

```
make
```

Po pomyślnej kompilacji zostanie utworzony plik wykonywalny. Aby uruchomić program w terminalu systemowym, należy użyć polecenia:

```
./system_rezerwacji
```